

AG3 - Algoritmo de Colonia de Hormigas (ACO)

Asignatura: Algoritmos de Optimización

Alumno: Ricardo Germes Serrano

URL Repositorio GitHub de las 3 Prácticas: <https://github.com/RGS70/03MIAR-Proyecto/tree/main/PR%C3%81CTICAS>

URL GitHub (ipynb): https://github.com/RGS70/03MIAR-Proyecto/blob/main/PR%C3%81CTICAS/AG3_Algoritmos_Colonia_de_Hormigas_Ricardo_Germes_Serrano.ipynb

URL GitHub (pdf): https://github.com/RGS70/03MIAR-Proyecto/blob/main/PR%C3%81CTICAS/AG3_Algoritmos_Colonia_de_Hormigas_Ricardo_Germes_Serrano.pdf

Este notebook resuelve el planteamiento de mejora que quedaba pendiente en `Add_Nodo`: en vez de elegir el siguiente nodo con probabilidad uniforme, se implementa la función de probabilidad basada en feromonas y distancia:

$$p_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha [\nu_{ij}]^\beta}{\sum_{l \in J_i^k} [\tau_{il}(t)]^\alpha [\nu_{il}]^\beta}$$

También se corrige un pequeño error de índices en `Evaporar_Feromonas` (transponía la matriz de feromonas en cada evaporación), necesario para que la mejora de `Add_Nodo` use correctamente los valores de feromona.

```
In [1]: #Modulo de llamadas http para descargar ficheros
!pip install requests
```

```
#Libreria del problema TSP: http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsplib.html
!pip install tsplib95
```

```
Requirement already satisfied: requests in C:\Datos\EV_MASTER\Lib\site-packages (2.34.2)
Requirement already satisfied: charset_normalizer<4,>=2 in C:\Datos\EV_MASTER\Lib\site-packages (from requests) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in C:\Datos\EV_MASTER\Lib\site-packages (from requests) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in C:\Datos\EV_MASTER\Lib\site-packages (from requests) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in C:\Datos\EV_MASTER\Lib\site-packages (from requests) (2026.5.20)
Requirement already satisfied: tsplib95 in C:\Datos\EV_MASTER\Lib\site-packages (0.7.1)
Requirement already satisfied: Click>=6.0 in C:\Datos\EV_MASTER\Lib\site-packages (from tsplib95) (8.4.2)
Requirement already satisfied: Deprecated~=1.2.9 in C:\Datos\EV_MASTER\Lib\site-packages (from tsplib95) (1.2.18)
Requirement already satisfied: networkx~=2.1 in C:\Datos\EV_MASTER\Lib\site-packages (from tsplib95) (2.8.8)
Requirement already satisfied: tabulate~=0.8.7 in C:\Datos\EV_MASTER\Lib\site-packages (from tsplib95) (0.8.10)
Requirement already satisfied: wrapt<2,>=1.10 in C:\Datos\EV_MASTER\Lib\site-packages (from Deprecated~=1.2.9->tsplib95) (1.17.3)
Requirement already satisfied: colorama in C:\Datos\EV_MASTER\Lib\site-packages (from Click>=6.0->tsplib95) (0.4.6)
```

```
In [2]: import tsplib95
import random
from math import e
import urllib.request
```

```
In [3]: #DATOS DEL PROBLEMA
#file = "swiss42.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/swiss42.tsp", "swiss42.tsp")
#!gzip -d swiss42.tsp.gz #Descomprimir el fichero de datos

# El fichero ya existe al descargarse en la práctica 3 (omitimos la descarga)
file = "swiss42.tsp"
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)
```

```
#Devuelve la distancia total de una trayectoria/solucion(lista de nodos)
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i],solucion[i+1] , problem)
    return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], problem)
```

Algoritmo de colonia de hormigas

La función Add_Nodo selecciona al azar un nodo con probabilidad uniforme. Para ser mas eficiente debería seleccionar el próximo nodo siguiendo la probabilidad correspondiente a la ecuación:

$$p^k_{ij}(t) = \frac{[\tau_{ij}(t)]^{\alpha} [\nu_{ij}]^{\beta}}{\sum_{i \in J^k_i} [\tau_{il}(t)]^{\alpha} [\nu_{il}]^{\beta}}, \text{ si } j \in J^k_i$$

$$p^k_{ij}(t) = 0, \text{ si } j \notin J^k_i$$

```
In [4]: def Add_Nodo_original(problem, H ,T ) :
    #Mejora:Establecer una funcion de probabilidad para
    # añadir un nuevo nodo dependiendo de los nodos mas cercanos y de las feromonas depositadas
    Nodos = list(problem.get_nodes())
    return random.choice( list(set(range(1,len(Nodos))) - set(H) ) )

def Add_Nodo(problem, H ,T, alpha=1, beta=3 ) :
    #Mejora resuelta: función de probabilidad para añadir un nuevo nodo dependiendo
    #de la cercanía (distancia) a los nodos candidatos y de las feromonas depositadas
    Nodos = list(problem.get_nodes())
    nodo_actual = H[-1]
    candidatos = list(set(range(1,len(Nodos))) - set(H))

    pesos = []
    for candidato in candidatos:
        tau = T[nodo_actual][candidato] #Cantidad de feromona en la arista (i,j)
        d = problem.get_weight(nodo_actual, candidato)
        nu = 1/d if d > 0 else 1e6 #Inversa de la distancia de i a j
        pesos.append((tau**alpha) * (nu**beta))

    total_pesos = sum(pesos)
    probabilidades = [p/total_pesos for p in pesos]

    #Elegimos el siguiente nodo al azar, pero orientado por la probabilidad p_ij
    return random.choices(candidatos, weights=probabilidades, k=1)[0]

def Incrementa_Feromona(problem, T, H ) :
    #Incrementa segun la calidad de la solución. Añadir una cantidad inversamente proporcional a la distancia tot
    for i in range(len(H)-1):
        T[H[i]][H[i+1]] += 1000/distancia_total(H, problem)
    return T

def Evaporar_Feromonas_original(T ) :
    #Evapora 0.3 el valor de la feromona, sin que baje de 1
    #Mejora:Podemos elegir diferentes funciones de evaporación dependiendo de la cantidad actual y de la suma tot
    T = [[ max(T[i][j] - 0.3 , 1) for i in range(len(Nodos)) ] for j in range(len(Nodos))]
    return T

def Evaporar_Feromonas(T ) :
    #Evapora 0.3 el valor de la feromona, sin que baje de 1
    #Mejora:Podemos elegir diferentes funciones de evaporación dependiendo de la cantidad actual y de la suma tot
    #(Se corrige aquí un pequeño error de índices del planteamiento original: la doble comprensión
    #recorría primero "i" y luego "j" al construir cada fila, lo que transponía la matriz de
    #feromonas en cada evaporación. Con la mejora de Add_Nodo, que sí usa el valor de T, es
    #importante que los índices T[i][j] se mantengan consistentes entre incremento y evaporación)
    T = [[ max(T[i][j] - 0.3 , 1) for j in range(len(Nodos)) ] for i in range(len(Nodos))]
    return T

In [5]: def hormigas(problem, N) :
    #problem = datos del problema
    #N = Número de agentes(hormigas)

    #Nodos
    Nodos = list(problem.get_nodes())
    #Aristas
    Aristas = list(problem.get_edges())

    #Inicializa las aristas con una cantidad inicial de feromonas:1
    #Mejora: inicializar con valores diferentes dependiendo diferentes criterios
    T = [[ 1 for _ in range(len(Nodos)) ] for _ in range(len(Nodos))]

    #Se generan los agentes(hormigas) que serán estructuras de caminos desde 0
```

```
Hormiga = [[0] for _ in range(N)]
```

```
#Recorre cada agente construyendo la solución
```

```
for h in range(N) :
```

```
#Para cada agente se construye un camino
```

```
for i in range(len(Nodos)-1) :
```

```
#Elige el siguiente nodo
```

```
Nuevo_Nodo = Add_Nodo(problem, Hormiga[h] ,T )
```

```
Hormiga[h].append(Nuevo_Nodo)
```

```
#Incrementa feromonas en esa arista
```

```
T = Incrementa_Feromona(problem, T, Hormiga[h] )
```

```
#print("Feromonas(1)", T)
```

```
#Evapora Feromonas
```

```
T = Evaporar_Feromonas(T)
```

```
#print("Feromonas(2)", T)
```

```
#Seleccionamos el mejor agente
```

```
mejor_solucion = []
```

```
mejor_distancia = 10e100
```

```
for h in range(N) :
```

```
distancia_actual = distancia_total(Hormiga[h], problem)
```

```
if distancia_actual < mejor_distancia:
```

```
mejor_solucion = Hormiga[h]
```

```
mejor_distancia =distancia_actual
```

```
print(mejor_solucion)
```

```
print(mejor_distancia)
```

```
hormigas(problem, 1000)
```

```
[0, 1, 4, 3, 2, 27, 30, 29, 9, 23, 41, 8, 10, 25, 18, 11, 12, 26, 5, 6, 7, 16, 15, 14, 13, 19, 37, 17, 31, 36, 3  
5, 33, 34, 20, 38, 22, 39, 21, 40, 24, 28, 32]  
1603
```

In []: